

Agent Runtime & Capability Gateway

Detailed architecture and implementation specification

United States deployment baseline

RUNTIME	Bounded cognitive execution from signed profiles and governed context
GATEWAY	Independent authorization, data controls, credentials, routing and evidence
SAFETY	Effect classes, fencing, layered idempotency, revocation and case-command validation
AZURE	Foundry hosted agents, Container Apps, APIM, Service Bus, PostgreSQL and Key Vault

Planning baseline | Version 1.0 | 17 September 2026

Defines the governed boundary between replaceable agent compute and foundation/KYC capabilities. Configuration values are proposed and require target-subscription validation.

Contents

Purpose and outcome	3
1. Mission, boundary, and invariants	3
2. Logical architecture	4
3. Trust model	5
4. Generic agent contract	5
5. AgentRequest envelope	6
6. ContextManifest and prompt construction	6
7. Runtime lifecycle	7
8. Reasoning loop and stop conditions	7
9. AgentResult and ExecutionRecord	7
10. Runtime portability	8
11. Foundry isolation and configuration	8
12. Capability model	9
13. CapabilityRequest contract	9
14. Capability Gateway request path	10
15. CapabilityResult contract	10
16. Effect classes and authorization strength	11
17. Idempotency and ambiguous outcomes	11
18. Model Gateway	12
19. Tool, connector, and MCP Gateway	12
20. Identity and credential brokerage	12
21. APIM configuration	13
22. Network architecture	13
23. Data protection and evidence	14
24. Reliability, retries, and backpressure	14
25. Cancellation, fencing, and revocation	14
26. Persistence model	15
27. Azure resource topology and sizing	16
28. Observability and flight recorder	16
29. Security threats and controls	17
30. KYC capability catalog baseline	18
31. CI/CD and release integration	18
32. Implementation plan	19
33. Prioritized engineering backlog	20
34. Verification strategy	20
35. Operational runbooks	21
36. Architecture decisions and definition of done	21

Purpose and outcome

The Agent Runtime and Capability Gateway execute bounded cognitive work without granting an agent direct authority over enterprise systems. The runtime receives a signed, resolved ExecutionProfile and a minimized ContextManifest. It may reason, call only compiled capabilities through the gateway, and return a typed AgentResult. The gateway independently authenticates the runtime, validates the PermissionEnvelope, applies data and effect controls, obtains downstream credentials, invokes approved providers, and returns normalized evidence.

This United States baseline extends the foundation, KYC domain, Harness Control Plane, and Execution Control Plane designs. It assumes separate non-production and production Azure subscriptions, US data residency, private networking, Microsoft Entra workload identities, Microsoft Foundry Agent Service as the primary managed agent substrate, Azure Container Apps as a portable runtime option, and Azure API Management as the primary capability ingress. All Azure sizing values are planning values that must be validated in target subscriptions.

1. Mission, boundary, and invariants

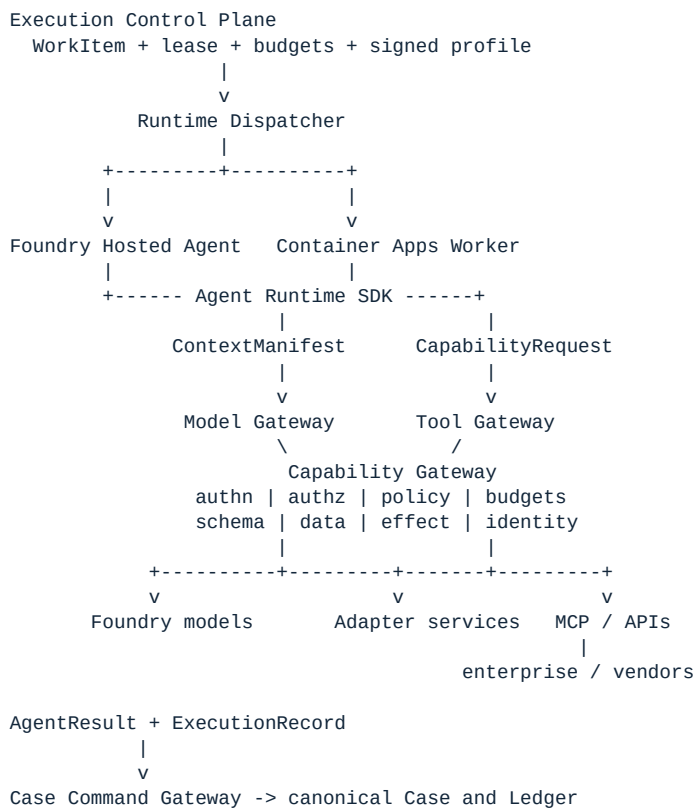
The Agent Runtime is replaceable cognitive compute. It does not own workflow state, case truth, policy truth, tool credentials, release pointers, or final business decisions. The Capability Gateway is the boundary between that untrusted compute and models, APIs, tools, MCP servers, enterprise connectors, and external providers.

The Execution Control Plane owns WorkItems, readiness, leases, deadlines, cancellation, budgets, retries, and completion. The Context Assembler owns the authorized data view. The Case Command Gateway owns acceptance of proposed business changes. The Harness Control Plane owns signed executable authority. The Agent Runtime and Capability Gateway consume those contracts and produce a complete computational record.

The following invariants are enforced:

- Every invocation references a released, signed, and non-revoked HarnessPlan digest and a resolved ExecutionProfile digest.
- An agent receives only the ContextManifest selected for that WorkItem and never reads the case database directly.
- An agent knows logical capability names; it never receives provider endpoints, API keys, client secrets, refresh tokens, or database credentials.
- Every model or tool call passes through the Capability Gateway or an equivalently governed Foundry tool path explicitly allowed by the plan.
- The gateway authorizes each call from compiled permission, case and tenant scope, purpose, data class, residency, effect class, budget, and current revocation state.
- Agent output is a proposal. Only the Case Command Gateway can make business state effective.
- Runtime conversation and framework state are noncanonical and disposable. The ExecutionRecord and referenced artifacts preserve the auditable computation.
- At-least-once execution never becomes duplicate business effect. Idempotency exists at the WorkItem, runtime, gateway, adapter, and provider boundaries.
- External content remains typed untrusted evidence and cannot become policy, system instruction, or authorization.
- Cancellation, deadline, budget exhaustion, lease fencing, or revocation stops further calls and prevents result acceptance.

2. Logical architecture



Component	Responsibility	Canonical state
Runtime dispatcher	Accept leased WorkItem, verify authority, choose execution stamp, start invocation	Invocation acceptance and dispatch reference only
Agent Runtime SDK	Validate envelope, run bounded reasoning loop, mediate model and tool calls, construct typed result	None; emits ExecutionRecord
Hosted agent or worker	Isolated compute for one invocation or session	Ephemeral session state only
Model gateway	Authorize model profile, apply token and content controls, route to approved deployment	Invocation and usage record
Tool gateway	Authorize capability, validate request, enforce effect semantics, route to adapter	Invocation, idempotency, and effect journal
Capability authorization service	Evaluate signed permission, context, policy, revocation, scope, and budget	Decision record with rule IDs
Credential broker	Acquire managed identity, OAuth, certificate, or secret-backed credential	No reusable credential returned to agent
Adapter service	Translate canonical capability contract to provider protocol and normalize response	Provider correlation and evidence references
Evidence writer	Store approved raw response and normalized evidence with digest and lineage	Immutable evidence artifact
Execution recorder	Assemble computational history and telemetry references	ExecutionRecord and artifact manifest

3. Trust model

Treat the agent, model, prompts, retrieved content, tool output, and remote MCP descriptions as untrusted inputs. Trust is conferred only by signed HarnessPlan authority, verified schemas, deterministic policy, authenticated identity, gateway enforcement, and accepted evidence provenance.

The runtime may decide how to analyze an allowed problem, which allowed capability to request, and when it has enough evidence. It may not expand its data access, select an undeclared provider, increase its budget, waive a policy obligation, construct credentials, or declare a business decision effective.

The Capability Gateway is a policy enforcement point, not the source of policy. It consumes a pinned PermissionEnvelope and current emergency controls. API Management handles transport-level authentication, routing, quotas, and policy. A gateway authorization service handles domain-specific permission, purpose, data-flow, effect, evidence, idempotency, and revocation checks that would be unsafe to encode only in portal configuration.

Input	Trust label	Permitted use
System instructions from signed plan	TRUSTED_INSTRUCTION	Runtime behavior within compiled authority
Context facts from accepted case state	TRUSTED_CASE_FACT	Reasoning and evidence correlation
Claims and findings	GOVERNED_ASSERTION	Reasoning with provenance and confidence
External documents and web text	UNTRUSTED_EXTERNAL	Evidence and analysis input only
Tool descriptions	SIGNED_CAPABILITY_METADATA	Discovery of already authorized operations
Model output	UNTRUSTED_PROPOSAL	Schema validation and downstream gate review
Tool response	PROVIDER_RESPONSE	Normalization and evidence validation

4. Generic agent contract

One runtime contract supports screening, investigation, evidence extraction, policy analysis, decision recommendation, and QA. Domain behavior comes from the resolved profile rather than framework-specific code.

```
execute_agent(  
  AgentRequest,  
  ExecutionProfile,  
  ContextManifest,  
  CancellationToken  
) -> AgentResult
```

An AgentSpec declares role, provided capabilities, model profile, input and output contracts, context selectors, allowed tools, skills, case commands, policies, limits, completion rules, and escalation triggers. The resolved ExecutionProfile binds all names to immutable versions and adds region, runtime stamp, model deployment class, tool provider bindings, evaluation attestation, PermissionEnvelope, budgets, and assurance.

The runtime implementation excludes vendor SDK selection, SQL, case writes, secrets, provider retry rules, policy definitions, workflow state transitions, and unbounded arbitrary code. Those concerns remain in gateway adapters and deterministic platform services.

5. AgentRequest envelope

The dispatcher sends a structured envelope instead of an arbitrary prompt. The runtime verifies the envelope signature, digest references, audience, deadline, lease fencing token, and revocation generation before loading any context.

```
{
  "contractVersion": "agent.request.v2",
  "execution": {
    "executionId": "exec_01...",
    "workItemId": "wi_01...",
    "caseId": "case_01...",
    "attempt": 2,
    "leaseId": "lease_01...",
    "fencingToken": 19,
    "deadlineAt": "2026-09-17T17:30:00Z",
    "traceparent": "00-..."
  },
  "authority": {
    "planDigest": "sha256:...",
    "executionProfileDigest": "sha256:...",
    "permissionEnvelopeDigest": "sha256:...",
    "policyDigest": "sha256:...",
    "revocationGeneration": 4812
  },
  "context": {
    "manifestDigest": "sha256:...",
    "expiresAt": "2026-09-17T17:30:00Z"
  },
  "budgets": {
    "maxRuntimeSeconds": 120,
    "maxModelCalls": 6,
    "maxToolCalls": 20,
    "maxInputTokens": 60000,
    "maxOutputTokens": 12000,
    "maxCostUsd": 1.50
  },
  "requiredOutput": "kyc.screening.result.v3"
}
```

The full case is never embedded. The envelope contains identifiers and content digests. The runtime retrieves the immutable ContextManifest through a short-lived, audience-bound authorization or receives it in the invocation only when size and transport policy allow.

6. ContextManifest and prompt construction

The Context Assembler produces a canonical manifest from the WorkItem, case snapshot, policy context, AgentSpec selectors, and PermissionEnvelope. Each item includes a schema, classification, trust label, purpose, provenance reference, content digest, and optional expiry.

The runtime converts this manifest into framework-specific messages through a deterministic prompt adapter. Trusted instructions and untrusted evidence occupy separate protocol fields where supported. When a framework has only text roles, the adapter wraps untrusted content in explicit data delimiters, escapes control sequences, and repeats that the content cannot authorize tools or modify instructions.

Prompt construction records prompt bundle digest, adapter version, item ordering, redaction actions, token estimate, and final message digest. Raw restricted content is not copied into logs. Reproduction uses the stored manifests and digests under controlled access.

Context overflow follows declared policy: deterministic prioritization, approved summarization with lineage, chunked subwork, or escalation. Silent truncation is prohibited. A summary is a new governed assertion with references to the source items; it never replaces raw evidence.

7. Runtime lifecycle

1. Authenticate the dispatcher and validate the AgentRequest schema.
2. Resolve and verify the signed HarnessPlan, ExecutionProfile, PermissionEnvelope, and revocation view.
3. Confirm WorkItem lease and fencing token with the Execution Control Plane.
4. Retrieve and digest-check the ContextManifest.
5. Initialize local budgets, cancellation, trace context, and append-only execution journal.
6. Build the model request from signed prompt bundle and governed context.
7. Send model calls through the Model Gateway.
8. Convert requested tool calls to typed CapabilityRequests; reject undeclared names locally.
9. Send each request through the Tool Gateway; consume normalized CapabilityResults.
10. Repeat within call, time, token, cost, and depth limits.
11. Validate the final result schema, evidence references, completion obligations, and unresolved contradictions.
12. Persist the ExecutionRecord and artifacts by digest.
13. Return AgentResult to the Execution Control Plane.
14. The Case Command Gateway independently validates any proposed commands before case-state mutation.

The runtime checks cancellation, deadline, lease, budget, and revocation before every model call, capability call, and final return. Local checks improve speed, but the gateway and result boundary recheck independently.

8. Reasoning loop and stop conditions

The runtime implements a bounded state machine rather than an open-ended chat loop.



The plan defines maximum iterations, model calls, tool calls, parallel calls, wall time, tokens, cost, repeated-call detection, and tool-call depth. The runtime stops when required outputs and evidence obligations are met, a declared escalation condition occurs, the model requests an unauthorized action, progress stalls, or any governing limit is reached.

Repeated identical calls with unchanged inputs are blocked after the configured threshold. The runtime detects self-delegation cycles and nested-agent depth. An orchestrator may propose more WorkItems but cannot spawn hidden background work outside the Execution Control Plane.

9. AgentResult and ExecutionRecord

AgentResult is small, typed, and safe for deterministic processing. It contains outcome class, structured findings, evidence references, contradictions, uncertainty, proposed case commands, escalation reason, budget usage, and the ExecutionRecord digest. Free-form explanation is optional supporting material, not the accepted business command.

```

{
  "contractVersion": "agent.result.v3",
  "executionId": "exec_01...",
  "status": "COMPLETED",
  "outputSchema": "kyc.screening.result.v3",
  "findings": [{"type": "sanctions", "classification": "NO_MATCH", "evidenceRefs": ["ev_..."]}],
  "contradictions": [],
  "proposedCommands": [{"type": "RecordScreeningEvidence", "payloadDigest": "sha256:..."}],
  "completion": {"mandatoryScreeningCompleted": true, "evidenceReferenced": true},

```

```
"usage": {"modelCalls":2, "toolCalls":3, "costUsd":0.42},
"executionRecordDigest": "sha256:..."
}
```

The ExecutionRecord binds the request, plan, profile, permission, context, prompt, model invocation, capability calls, normalized results, artifacts, output, runtime image, framework, region, timing, budget, and telemetry correlation. It preserves computational provenance without claiming that every internal token or provider implementation detail is durable business truth.

10. Runtime portability

The primary managed option is Microsoft Foundry Agent Service. Microsoft documents prompt agents and hosted agents; hosted agents accept custom code or containers and provide a managed endpoint, scaling, identity, session state, and observability: [Foundry Agent Service overview](#). Use hosted agents for the generic runtime when custom contracts, cancellation, budget logic, and gateway mediation are required.

Azure Container Apps workers implement the same /invoke contract as a portability and special-workload path. This path supports deterministic worker lifecycle, direct Service Bus scaling, framework independence, and workloads that cannot use a Foundry-hosted feature. Container Apps scale rules can authenticate to Azure queues and Service Bus with managed identity: [Container Apps scaling](#).

Both implementations must pass the same conformance suite. The ExecutionProfile selects a runtime stamp, not the workflow. Runtime choice cannot weaken PermissionEnvelope, network, evaluation, logging, or result-gate requirements.

11. Foundry isolation and configuration

Use separate Foundry projects for production and non-production, and partition production when residency, contractual isolation, tenant scale, model availability, or risk requires it. A project is a security and lifecycle boundary, not merely a folder.

For regulated production, use a network-isolated Foundry configuration. Microsoft documents BYO VNet flows in which hosted-agent sessions run in isolated compute, tool calls route through a project data proxy, and customer resources are reached through private endpoints: [Foundry networking deep dive](#). The deployment must validate tool support because not every tool or destination has identical private-network behavior.

Planning configuration:

Setting	Production baseline	Reason
Region	East US 2 primary; approved US recovery region	Align runtime, data, and model processing
Project model	Regulated shared plus tenant-dedicated projects where required	Balance isolation and operations
Hosted-agent subnet	/24 initial per high-scale project; measure session mapping	Absorb sessions, revisions, upgrades, and headroom
Subnet utilization	Alert at 70 percent; keep below 80 percent	Preserve rollout and failure headroom
ACR	Private Premium registry, images by digest	Controlled agent image supply chain
Public network access	Disabled after private connectivity validation	Prevent ungoverned paths
Agent identity	Dedicated Entra identity per deployed agent	Least privilege and attribution
Telemetry	OpenTelemetry to dedicated Application Insights	End-to-end execution correlation

Foundry session state is not the canonical WorkItem, case, or execution record. A lost session can be reconstructed from the AgentRequest and governed artifacts. New agent versions are immutable dependencies pinned in the ExecutionProfile.

12. Capability model

A capability is a stable business operation with one canonical request and response contract. Providers are replaceable implementations. Examples include `identity.document.verify`, `identity.ssn.verify`, `screening.sanctions.search`, `screening.pep.search`, `adverse_media.search`, `customer.internal.search`, `model.reasoning.standard`, and `case.command.propose`.

```
apiVersion: harness.factory/v1
kind: CapabilityVersion
metadata:
  name: screening.sanctions.search
  version: 3.2.0
spec:
  requestSchema: kyc.sanctions.request.v2
  responseSchema: kyc.sanctions.result.v3
  effectClass: READ_ONLY
  data:
    accepts: [PII]
    returns: [PII, SCREENING_RESULT]
    residency: US
    egress: EXTERNAL_APPROVED
  reliability:
    timeoutMs: 8000
    attempts: 3
    retryOn: [429, 502, 503, 504]
  evidence:
    produces: screening_result
    rawResponseRetention: required
  telemetry:
    payloadLogging: prohibited
```

The Capability Resolver in the Execution Control Plane selects an eligible provider binding using region, tenant, data class, assurance, evaluation certification, provider health, contract, cost, and SLO. The gateway validates that binding and may fail over only within a precompiled equivalent-provider set. It does not invent a provider because one appears healthy.

13. CapabilityRequest contract

```
{
  "contractVersion": "capability.request.v2",
  "invocationId": "cinv_01...",
  "executionId": "exec_01...",
  "workItemId": "wi_01...",
  "caseId": "case_01...",
  "tenantId": "tenant_01...",
  "capability": "screening.sanctions.search",
  "capabilityVersion": "3.2.0",
  "providerBindingDigest": "sha256:...",
  "permissionEnvelopeDigest": "sha256:...",
  "purpose": "KYC_SCREENING",
  "dataClasses": ["PII"],
  "effectClass": "READ_ONLY",
  "deadlineAt": "2026-09-17T17:29:10Z",
  "idempotencyKey": "tenant_01:case_01:wi_01:sanctions:subject_01",
  "inputSchema": "kyc.sanctions.request.v2",
  "input": {"subjectRef": "subject_01", "name": "...", "dateOfBirth": "..."},
  "traceparent": "00-..."
}
```

The runtime signs or authenticates the request with its Entra workload identity. The gateway derives agent and runtime identity from the token rather than accepting identity fields as proof. It resolves the PermissionEnvelope by digest and compares the request with allowed capability, input fields, data classes, purpose, region, effect, provider set, deadlines, and remaining budget.

14. Capability Gateway request path

1. APIM authenticates the Entra token, validates issuer, audience, tenant, subject, and required claims.
2. APIM applies network restrictions, request size, media type, schema, rate, concurrency, and basic quota policy.
3. The gateway service verifies the signed plan and PermissionEnvelope, release scope, revocation generation, lease, deadline, and fencing token.
4. The authorization engine evaluates capability, version, provider binding, purpose, data classes, residency, tenant, assurance, effect class, and budget.
5. The idempotency service returns an existing terminal result or atomically reserves the operation.
6. The data guard validates allowed fields, tokenizes or redacts where specified, and writes a request digest.
7. The credential broker obtains the provider identity without exposing it to the runtime.
8. The adapter invokes the provider using capability-specific timeout, retry, circuit, and concurrency rules.
9. The response is schema-validated, normalized, classified, and checked for provider errors or unsafe content.
10. Required raw response and normalized evidence are stored with digest and provenance.
11. The invocation and effect journal are committed; the gateway returns a typed CapabilityResult.

APIM provides strong gateway primitives, including Entra token validation, managed-identity backend authentication, rate limiting, backend pools, and circuit-breaker configuration. The domain authorization and effect journal remain application services because they require authoritative Harness and case context.

15. CapabilityResult contract

```
{
  "contractVersion": "capability.result.v2",
  "invocationId": "cinv_01...",
  "status": "SUCCEEDED",
  "capability": "screening.sanctions.search",
  "providerBindingDigest": "sha256:...",
  "outputSchema": "kyc.sanctions.result.v3",
  "output": {"matchClass": "NO_MATCH", "candidates": []},
  "evidence": [{"evidenceId": "ev_01...", "digest": "sha256:...", "type": "screening_result"}],
  "authorizationDecisionId": "authz_01...",
  "effectJournalId": "eff_01...",
  "providerCorrelationHash": "sha256:...",
  "usage": {"durationMs": 412, "billableUnits": 1},
  "cache": {"status": "MISS", "freshUntil": "2026-09-17T18:00:00Z"}
}
```

Errors use stable classes: UNAUTHORIZED, REVOKED, INVALID_INPUT, DATA_POLICY_DENIED, BUDGET_EXCEEDED, DEADLINE_EXCEEDED, RATE_LIMITED, PROVIDER_UNAVAILABLE, PROVIDER_REJECTED, INVALID_PROVIDER_RESPONSE, AMBIGUOUS_EFFECT, and INTERNAL_ERROR. The runtime uses the class and compiled retry policy; it never infers retry behavior from free-form text.

16. Effect classes and authorization strength

Effect class	Examples	Required controls
READ_ONLY	Search sanctions, retrieve approved record	Standard permission, bounded retries, evidence and freshness
IDEMPOTENT_WRITE	Upsert vendor case by stable key	Mandatory idempotency key, provider support or gateway effect journal
REVERSIBLE_WRITE	Add watchlist subscription that can be removed	Explicit command, compensation definition, stronger approval
IRREVERSIBLE_WRITE	Submit regulatory filing	Human or deterministic approval gate, no blind retry, receipt reconciliation
EXTERNAL_COMMUNICATION	Send customer notice or analyst email	Approved template, recipient control, content policy, preview and approval
FINANCIAL_OR_LEGAL_EFFECT	Freeze account, reject customer, file SAR-related action	Separate business command, dual control, strict evidence and audit

Agents are normally restricted to read-only capabilities and proposals. A plan may authorize a stronger effect only when the workflow includes the required approval, compensation, evidence, provider idempotency, and result reconciliation. The runtime cannot upgrade an effect class by choosing a different endpoint or MCP tool alias.

17. Idempotency and ambiguous outcomes

The gateway stores an idempotency record before invoking any billable or effecting operation. Its key scope includes tenant, capability, provider binding, business operation key, and relevant request digest. A duplicate with identical input returns the prior terminal result. A duplicate key with different input fails as a conflict.

For provider timeout after request transmission, the operation becomes `OUTCOME_UNKNOWN`. Read-only calls may retry when the plan permits. Writes enter reconciliation: query provider status by idempotency key or correlation reference, wait, compensate, or route to human review. The gateway never reports success solely because the HTTP request was accepted.

```
create table capability_invocation (
  invocation_id uuid primary key,
  execution_id uuid not null,
  capability text not null,
  provider_binding_digest text not null,
  request_digest text not null,
  idempotency_scope text not null,
  state text not null,
  authorization_decision_id uuid not null,
  effect_class text not null,
  provider_correlation_hash text,
  result_digest text,
  attempt_count integer not null,
  deadline_at timestampz not null,
  row_version bigint not null default 0,
  unique (idempotency_scope)
);
```

```
create table effect_journal (
  effect_journal_id uuid primary key,
  invocation_id uuid not null unique,
  effect_class text not null,
  state text not null,
  requested_at timestampz not null,
  acknowledged_at timestampz,
  reconciled_at timestampz,
  receipt_artifact_digest text,
  compensation_invocation_id uuid
);
```

18. Model Gateway

The Model Gateway is a specialization of the Capability Gateway. The agent requests a logical profile such as `model.reasoning.standard`; the `ExecutionProfile` binds it to an approved deployment and fallback set. The gateway enforces model, region, content, data-class, token, cost, request-rate, and concurrency policy.

Azure API Management provides AI gateway capabilities for models, agents, remote MCP servers, and other tools across API Management tiers: [AI gateway capabilities](#). The dedicated AI Gateway tier is public preview and region-limited as of September 2026, so this regulated baseline uses a generally available APIM tier and only GA policies in the production critical path. Preview features may be evaluated in non-production.

The gateway records model profile, provider and deployment identifiers, API version, sampling configuration, prompt and response digests, content-safety decisions, token counts, latency, cost estimate, cache state, and fallback. Raw prompts and outputs follow the `ContextManifest` classification and are not placed in APIM request logs.

Fallback is compiled, not opportunistic. It requires equivalent data handling, region, interface, evaluation certification, and assurance. A lower-cost or available model is not a valid fallback if the exact agent-model-tool combination lacks the required attestation.

19. Tool, connector, and MCP Gateway

Enterprise connectors sit behind adapters and APIM. Agents do not call SAP, Salesforce, ServiceNow, SharePoint, Microsoft Graph, core banking, customer master, document management, email, or vendor APIs directly. An adapter limits the operation set, translates schemas, applies provider-specific identity, and normalizes evidence.

MCP is a transport and discovery protocol, not authorization. API Management can expose a managed REST API as an MCP server or govern an existing MCP server: [MCP servers in API Management](#). The gateway publishes only approved tools, pins server and schema versions where possible, validates every invocation against the `PermissionEnvelope`, and ignores remote tool descriptions as sources of authority.

A capability name maps to a controlled APIM operation or MCP tool identifier. Namespace collisions, dynamic remote tool discovery, unapproved server redirects, schema changes, and tool-list expansion fail closed. The Harness Control Plane reevaluates and releases any executable tool catalog change.

20. Identity and credential brokerage

Five principal classes remain separate: human operators, Harness platform services, agent identities, gateway and adapter identities, and delegated end-user identities. Each hosted agent or runtime deployment gets a dedicated Entra identity. APIM validates its token and passes verified identity context to the gateway service.

The credential order is managed identity, workload federation or on-behalf-of token, client certificate, then secret. Secrets that cannot be eliminated live in Key Vault behind private endpoints. An agent never sees the downstream credential. The gateway or adapter acquires it immediately before the call and sends it only to the intended audience.

Delegated user authority is used only when the downstream action genuinely requires that user. The `PermissionEnvelope` must allow delegation, the original user token must have the required authentication context, and the gateway obtains an audience-specific on-behalf-of token. A service identity cannot silently impersonate a user, and a user token cannot expand the agent's compiled authority.

RBAC roles separate APIM policy administration, capability registry changes, adapter deployment, credential configuration, runtime invocation, production diagnostics, and audit. Anyone able to edit policies that use APIM managed identity is treated as privileged because such a policy can exercise that identity.

21. APIM configuration

Use a production APIM tier that provides the required private-network, availability, scale, and policy features in the selected region. Choose the exact tier after performance and feature validation; do not make the production critical path depend on the preview-only AI Gateway tier.

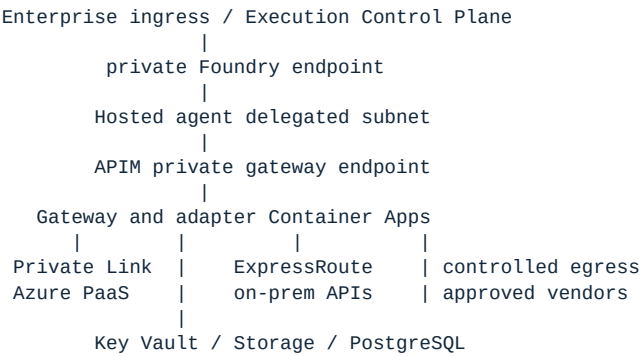
Policy layer	Controls
Global	Correlation ID, payload size, allowed TLS, generic headers, telemetry redaction, default deny
Product or workspace	Runtime identity groups, environment, tenant partition, quotas
API	Entra token validation, audience, capability family, schema version
Operation	Capability and effect metadata, request schema, deadline, rate and concurrency limit
Backend	Managed identity or credential manager, timeout, connection, load pool, circuit breaker
Outbound	Response size and schema, forbidden headers, classification, correlation
Error	Stable error mapping, retry hints, sanitized diagnostics

Use `validate-azure-ad-token` at API or operation scope for Microsoft Entra tokens: [Validate Microsoft Entra token](#). Backend entities centralize routing and can use pools and circuit breakers: [Set backend service](#). IaC owns APIs, named values, policies, backends, products, diagnostics, loggers, private networking, and role assignments. Portal edits in production are denied or continuously detected.

APIM metadata logging includes request ID, capability, version, identity subject hash, tenant hash, decision ID, provider binding digest, response class, latency, token or billable units, and trace context. Payload and authorization tokens are excluded.

22. Network architecture

The runtime and gateway use private network paths wherever supported. The production topology contains a Foundry delegated subnet, private endpoint subnet, Container Apps infrastructure subnet, APIM subnet where required by the tier, adapter subnet or environment, and controlled egress through Azure Firewall and NAT Gateway.



Private DNS zones are linked centrally. Network security groups permit only documented flows. Azure Firewall application rules allow approved vendor domains and Azure dependencies; IP rules are used where FQDN control is insufficient. DNS, firewall, APIM, and gateway logs correlate with execution IDs without storing payloads.

Foundry tool traffic may route through its project data proxy depending on the selected integration. Direct runtime egress is still denied for enterprise tools; the agent calls the Capability Gateway. Network reachability never substitutes for application authorization.

23. Data protection and evidence

The `PermissionEnvelope` specifies allowed input and output data classes, purpose, processing region, provider class, retention, and egress. The gateway compares those declarations with the capability and provider binding before the call. It rejects combinations such as US-only evidence with an EU-only endpoint or restricted PII with a model profile that lacks permission.

Request transformations remove fields the provider does not need. Stable references replace raw identifiers when an adapter can resolve them internally. Logs hash or tokenize tenant, case, subject, and provider references. Diagnostics expose classifications and digests rather than raw values.

Raw provider responses are retained only when evidence policy requires them. They are written to Blob storage with content digest, provider correlation, request digest, collection time, classification, retention, legal-hold eligibility, and encryption context. The normalized `CapabilityResult` references the evidence and records the transformation version.

Cache entries are permitted only for capabilities with declared cache semantics. The cache key binds tenant, provider, input digest, purpose, data policy, capability version, and freshness class. Restricted cross-tenant caching is prohibited. Revocation and provider-data corrections can invalidate entries by tag.

24. Reliability, retries, and backpressure

The runtime is stateless between invocations except for disposable framework session state. It checkpoints only at governed boundaries: model invocation, capability result, and final result. A worker crash causes the Execution Control Plane to retry the `WorkItem` with the same business idempotency keys and a new execution attempt.

The gateway owns provider-specific retry policy. It honors the caller deadline, subtracts elapsed time, uses exponential backoff with jitter, and never retries beyond the compiled attempt or cost budget. Circuit breakers protect providers. Bulkheads isolate model, screening, search, document, and enterprise connector pools. Tenant quotas prevent a noisy tenant from exhausting shared capability capacity.

Service Bus remains at-least-once transport even when duplicate detection is enabled. Microsoft documents application-controlled message IDs and a duplicate-detection window: [Service Bus duplicate detection](#). Business idempotency remains authoritative.

Backpressure begins at the dispatcher. Queue age, provider concurrency, APIM capacity, subnet IP availability, model quotas, token rate, and cost budget feed admission control. Low-priority investigations pause before high-priority sanctions or analyst-response work is affected.

25. Cancellation, fencing, and revocation

The runtime receives a cancellation channel and polls the Execution Control Plane at bounded intervals. Every gateway call carries execution, lease, attempt, and fencing data. The gateway rejects a stale token before invoking a provider. Result acceptance performs the same check after the call, preventing a timed-out worker from publishing late work.

Revocation is checked at invocation start, before each model or capability call, and before final result return. A new revocation generation invalidates cached authorization. Maximum propagation staleness is 60 seconds. High-assurance work fails closed when revocation status cannot be refreshed inside that bound.

For an in-flight irreversible effect, cancellation does not claim the external action vanished. The gateway records the actual state, attempts configured compensation or reconciliation, and returns `AMBIGUOUS_EFFECT` or the verified outcome. The `WorkItem` routes to human review when required.

26. Persistence model

Runtime metadata and gateway operations use separate PostgreSQL schemas from the canonical case. Large prompts, responses, raw evidence, and execution artifacts live in Blob storage and are referenced by digest.

```
create table runtime.execution_record (  
  execution_id uuid primary key,  
  work_item_id uuid not null,  
  attempt integer not null,  
  plan_digest text not null,  
  profile_digest text not null,  
  permission_digest text not null,  
  context_manifest_digest text not null,  
  runtime_image_digest text not null,  
  state text not null,  
  result_digest text,  
  artifact_manifest_digest text,  
  started_at timestamptz not null,  
  completed_at timestamptz,  
  usage jsonb not null,  
  unique (work_item_id, attempt)  
);  
  
create table gateway.authorization_decision (  
  decision_id uuid primary key,  
  invocation_id uuid not null,  
  subject_id_hash text not null,  
  permission_digest text not null,  
  capability text not null,  
  provider_binding_digest text not null,  
  outcome text not null,  
  reason_codes text[] not null,  
  policy_digest text not null,  
  revocation_generation bigint not null,  
  decided_at timestamptz not null  
);
```

State transitions use optimistic concurrency and an outbox in the same transaction. The evidence writer registers an artifact only after upload, server-side encryption, digest verification, malware controls where applicable, and required metadata succeed.

27. Azure resource topology and sizing

Azure resource	Production baseline	Initial configuration
Foundry resource and projects	East US 2; private networking	Hosted-agent runtime, dedicated identities, private ACR, project isolation
APIM	GA production tier with required VNet and availability features	Private gateway path, zones where supported, autoscale/capacity alerts, payload logging off
Container Apps environment	Workload profiles, zone redundant where supported	Runtime alternative, gateway auth service, adapters, recorder; internal ingress
Service Bus Premium	Namespace per environment	Work queues, cancellation and control topics, DLQ, private endpoint, managed-identity scaler
PostgreSQL Flexible Server	Separate runtime/gateway database or schemas	Zone-redundant HA, Entra auth, private access, PITR
Storage accounts	Separate evidence and runtime-artifact accounts	Private endpoints, versioning, soft delete, lifecycle and immutability policies
Key Vault Premium	Vault per environment and purpose	Provider secrets/certificates only when identity cannot replace them; purge protection
ACR Premium	Registry per environment	Private endpoint, image digests, OCI signing, vulnerability gate
App Configuration	Environment and region store	Non-secret gateway routing projections and feature controls; private endpoint
Azure Monitor	Dedicated App Insights and Log Analytics	OTel, redaction, sampling rules, alerts, diagnostic settings
Azure Firewall and NAT Gateway	Central controlled egress	FQDN rules, threat intelligence, stable vendor source IP where required

Initial Container Apps settings are runtime workers min 0 and max 100 per workload class, gateway authorization service min 3 and max 50, adapter services min 2 for critical providers, and recorder min 2. Scale on Service Bus queue depth, HTTP concurrency, and custom latency where supported. The production Foundry subnet starts at /24 for a high-scale project and is recalculated from tested concurrent sessions, revision overlap, data proxy needs, and 20 percent headroom.

Example resource groups:

```
rg-harness-agent-runtime-eus2
rg-harness-capability-gateway-eus2
rg-harness-runtime-data-eus2
rg-harness-runtime-security-eus2
rg-harness-runtime-observability-us
```

28. Observability and flight recorder

OpenTelemetry spans follow the WorkItem through dispatch, context load, prompt construction, model calls, capability authorization, adapter calls, evidence writes, result validation, and case-command submission. The trace ID is correlation, not business identity.

Required semantic attributes include environment, region, plan digest, profile digest, execution ID, WorkItem ID, attempt, agent version, runtime stamp, model profile, capability, provider binding digest, policy digest, assurance, outcome, error class, latency, token counts, tool counts, billable units, and cost estimate. Tenant and case identifiers are hashed or access-controlled.

Objective	Proposed target	Alert
Runtime invocation availability	99.9 percent monthly	Multi-window burn rate

Objective	Proposed target	Alert
Gateway availability	99.95 percent monthly	Multi-window burn rate
Gateway authorization latency	p95 under 40 ms excluding provider	p95 above 75 ms for 15 minutes
Read-only capability overhead	p95 under 100 ms excluding provider	p95 above 175 ms
Revocation enforcement	99.9 percent under 60 seconds	Any resolver stale over 60 seconds
Unauthorized capability denial	100 percent	Any accepted unauthorized test or production event
Duplicate external effects	Zero	Any effect-journal reconciliation mismatch
Execution record completeness	100 percent terminal executions	Reconciliation finds missing manifest

Dashboards separate platform reliability, model behavior, tool behavior, policy outcomes, KYC business outcomes, and human-review consistency. A single agent-accuracy metric cannot explain the system.

29. Security threats and controls

Threat	Primary controls
Prompt injection in documents or web results	Trust labels, context separation, gateway authorization independent of prompt, adversarial tests
Tool-name or schema spoofing	Signed capability catalog, exact version, schema digest, namespace and provider binding validation
Credential theft	Managed identity, gateway brokerage, no secret in context or response, Key Vault private access
Data exfiltration	PermissionEnvelope, field allowlists, provider classification, private network, controlled egress, payload redaction
Excessive agency	Bounded loop, budgets, effect classes, approval gates, no direct case writes
Cross-tenant access	Tenant-bound tokens, context manifests, database row controls, cache partitioning, adapter validation
Replay and duplicate effects	Nonces where required, idempotency scope, effect journal, provider reconciliation, fencing
Stale worker result	Lease and fencing checks at gateway and Case Command Gateway
Compromised provider or MCP server	Signature and schema pinning, response validation, evidence quarantine, revocation and provider isolation
Supply-chain compromise	Pinned images, SBOM, OCI signature, vulnerability gates, restricted build provenance
Telemetry leakage	Attribute allowlist, payload logging off, DLP tests, access-controlled diagnostics
Budget denial of service	Admission control, per-tenant/model/tool quotas, concurrency bulkheads, cost reservations

30. KYC capability catalog baseline

Capability	Effect	Key controls	Failure behavior
identity.document.verify	Read-only, billable	PII and document class, US processing, vendor consent, raw-response evidence	Retry bounded; manual review on inconclusive or outage
identity.ssn.verify	Read-only, billable	SSN minimization/tokenization, strict purpose, field allowlist	No fallback outside approved US providers
screening.sanctions.search	Read-only	Freshness, list version, alias inputs, match evidence	Approved equivalent-provider failover or manual review
screening.pep.search	Read-only	Jurisdiction and data-source version, match rationale	Manual review for high-confidence match
adverse_media.search	Read-only external	Public-data-only query, untrusted content labels, source provenance	Escalate if evidence insufficient
customer.internal.search	Read-only internal	Tenant scope, case purpose, security filters, no cross-customer bulk export	Deny on scope ambiguity
investigation.web.search	Read-only external	Query DLP, approved provider, result trust labeling, domain controls	Deny restricted identifiers or prohibited domain
document.ocr.extract	Read-only processing	Approved storage reference, malware scan, model profile and retention	Quarantine malformed document
notification.customer.preview	Proposal only	Approved template and recipient reference; no send effect	Return preview for deterministic approval
regulatory.submission.propose	Proposal only	Evidence and dual-control requirements	Never submit directly from agent runtime

31. CI/CD and release integration

The runtime SDK, gateway services, and adapters are versioned independently but certified as compatible combinations. Images are built once, scanned, given an SBOM, signed, and promoted by digest. The Harness Control Plane records runtime and adapter digests in evaluation attestations and ExecutionProfiles.

```
pull request
-> unit + contract + policy tests
-> prompt-injection and schema fuzz tests
-> build image + SBOM + vulnerability gate + OCI signature
-> deploy isolated nonprod by digest
-> runtime/gateway conformance suite
-> provider sandbox + fault + idempotency tests
-> KYC golden/adversarial/replay evaluations
-> performance and network validation
-> update Harness dependency + evaluation attestation
-> canary ExecutionProfile release
-> staged production promotion
```

APIM configuration, products, policies, backends, named-value references, loggers, diagnostics, and networking are IaC. Capability catalog records and provider bindings flow through Harness release controls. A portal hotfix requires incident authority, is exported to source immediately, and triggers drift reconciliation.

32. Implementation plan

Phase	Duration	Deliverables and exit criteria
0. Contracts and threats	2 weeks	AgentRequest, AgentResult, ExecutionRecord, CapabilityRequest/Result, effect classes, threat model, ADRs
1. Azure foundation	3 weeks	Foundry project, delegated subnet, APIM, Container Apps, Service Bus, PostgreSQL, Storage, Key Vault, ACR, monitoring
2. Runtime SDK MVP	5 weeks	Envelope verification, context adapter, bounded loop, cancellation, budgets, model/tool client, typed result
3. Gateway core	5 weeks	APIM ingress, authorization service, permission/revocation checks, schema guard, idempotency, evidence and errors
4. Model gateway	3 weeks	Logical profiles, routing, quotas, token/cost controls, content policy, telemetry, fallback enforcement
5. KYC adapters	5 weeks	Identity, SSN, sanctions, PEP, adverse media, internal search, OCR adapters and sandbox tests
6. Foundry hosted runtime	3 weeks	Container deployment, private network, identities, invoke contract, telemetry, revision and capacity tests
7. Effect and credential hardening	3 weeks	Credential broker, OBO, effect journal, ambiguous-outcome reconciliation, compensation hooks
8. End-to-end KYC pilot	4 weeks	Screening and investigation flows, case-command gating, replay, shadow, canary, human review
9. Resilience and DR	3 weeks	Load, chaos, provider outage, region exercise, runbooks, SLO alerts, security review

33. Prioritized engineering backlog

ID	Work item	Acceptance criterion
AR-01	Publish runtime protocol and schemas	Foundry and Container Apps implementations pass the same conformance fixtures
AR-02	Verify signed invocation authority	Altered digest, scope, audience, deadline, lease, or revocation fails before context load
AR-03	Implement governed context adapter	Trust labels and classifications survive prompt construction; silent truncation impossible
AR-04	Build bounded reasoning state machine	Limits, cancellation, stall detection, and stop rules pass deterministic tests
AR-05	Create complete ExecutionRecord	Every model/tool call and artifact is digest-bound without restricted payload leakage
CG-01	Configure private APIM ingress	Only approved identities and networks reach capability APIs
CG-02	Implement authorization engine	Permission, purpose, data, region, effect, budget, lease, and revocation are independently checked
CG-03	Implement idempotency and effect journal	Duplicate calls return stable result; ambiguous writes reconcile without duplicate effect
CG-04	Build credential broker	Agent never receives provider credential; managed identity is preferred and audited
CG-05	Implement schema and data guard	Extra fields, wrong classes, and prohibited provider combinations fail closed
CG-06	Build model-profile gateway	Logical profile routes only to attested deployment with token and cost controls
CG-07	Deliver sanctions and PEP adapters	Canonical results, evidence, retries, freshness, and provider errors pass KYC tests
CG-08	Deliver identity and SSN adapters	Restricted data is minimized, US-bound, and absent from telemetry
CG-09	Govern MCP exposure	Only released tools and schemas are visible and invocable through APIM
OPS-01	Implement revocation propagation	Runtime and gateway deny a revoked subject within 60 seconds
OPS-02	Add end-to-end OTel and dashboards	One trace follows dispatch through case-command result with redacted metadata
OPS-03	Run capacity and subnet tests	Two-times forecast peak meets latency, IP, quota, and cost targets
KYC-01	Execute production-like pilot	Golden, adversarial, fault, replay, shadow, and canary gates pass

34. Verification strategy

Unit tests cover envelopes, signatures, schema validation, context ordering, budget accounting, stop conditions, permission decisions, effect classes, idempotency, error mapping, retry rules, cache partitioning, and evidence manifests. Property tests generate equivalent call orderings and duplicate deliveries. Fuzzing targets JSON, MCP schemas, provider responses, prompt boundaries, and oversized inputs.

Integration tests use real non-production APIM, Foundry hosted agents, Container Apps, Service Bus, PostgreSQL, Blob, Key Vault, ACR, and provider sandboxes. They verify private DNS, identity audiences, policy deployment, token expiry, cancellation, scale, digest checking, evidence writes, and complete traces.

Adversarial tests place instructions in OCR, documents, search results, tool descriptions, tool output, and schema fields. They request undeclared capabilities, alternate endpoints, stronger effects, extra PII fields, cross-tenant references, stale

fencing tokens, and revoked providers. Every attempt must fail at a deterministic boundary with an audit decision.

Fault tests terminate runtime instances, duplicate messages, expire leases, delay revocation, exhaust model quota, fail APIM, time out adapters, return malformed provider data, lose a response after an external write, block evidence storage, and fail database over. Success means no authority expansion, no duplicate effect, no accepted stale result, and recoverable evidence.

Performance tests cover peak WorkItem bursts, long-running investigations, parallel tool calls, APIM capacity, authorization latency, provider pools, PostgreSQL connections, Service Bus backlog, Blob throughput, Foundry sessions, delegated-subnet IP use, and per-tenant fairness.

35. Operational runbooks

Required runbooks cover runtime backlog, Foundry capacity or subnet exhaustion, agent revision failure, context retrieval failure, model quota exhaustion, token or cost anomaly, APIM capacity, authorization latency, provider circuit open, connector credential failure, Key Vault outage, duplicate-effect alarm, ambiguous write, evidence-write failure, malformed provider response, MCP catalog drift, revocation lag, stale lease rejection, database failover, DLQ replay, and regional recovery.

Each runbook states detection, affected capabilities and cases, immediate containment, diagnostic queries, safe replay conditions, rollback or revocation threshold, evidence preservation, communications, and completion verification. Operators do not bypass the gateway, issue vendor calls from personal tools, expose credentials to the runtime, or edit effect-journal rows directly.

36. Architecture decisions and definition of done

The initial decisions approve a generic runtime contract; Foundry hosted agents as primary managed substrate; Container Apps as portable alternative; agents as untrusted compute; APIM plus domain gateway services as the capability boundary; compiled PermissionEnvelope as runtime authority; separate model and tool policies under one capability abstraction; gateway-owned credentials; typed trust labels; effect-class controls; at-least-once transport with layered idempotency; and Case Command Gateway validation before business mutation.

Production pilot readiness requires all of the following:

- Foundry and Container Apps runtimes pass the same AgentRequest and AgentResult conformance suite.
- An agent cannot access case storage, provider endpoint, model deployment, enterprise connector, or credential outside the gateway path.
- Permission, data, residency, purpose, assurance, effect, budget, lease, and revocation checks independently deny invalid calls.
- Every successful capability call has a decision record, request and result digest, provider binding, evidence references, usage, and trace correlation.
- Duplicate and ambiguous provider operations cannot produce an untracked duplicate business effect.
- Prompt injection in any context or tool field cannot authorize a capability, alter policy, or write case state.
- Revocation fences model and tool calls in under 60 seconds and stale worker results cannot pass the Case Command Gateway.
- KYC identity, SSN, sanctions, PEP, adverse-media, internal-search, and OCR capabilities pass contract, golden, adversarial, fault, replay, and canary tests.
- Private networking, managed identities, logging redaction, backup, recovery, APIM policy drift, subnet capacity, and provider outage runbooks are validated.
- Platform, security, privacy, KYC policy, SRE, audit, and operations owners approve evidence and ownership.

The result is a runtime in which agents can reason flexibly while every meaningful interaction remains bounded by compiled authority, minimized context, independent gateway enforcement, explicit effect semantics, controlled credentials, durable evidence, and deterministic business acceptance.